🛠️ CST 182.2 GUI TICKET MANAGER

Contents

Project Brief

Project Repo Link

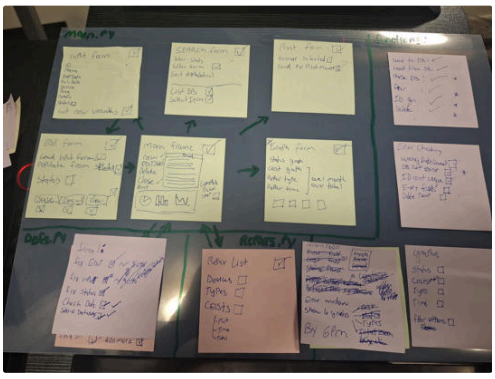
GitHub - CaffeinatedAndroid/CST182.2-TICKET-MANAGER-GUI

Selecting a Project

The program that was chosen was a Ticket Manager that has a GUI and some extra functions such as exporting to a text document, sending to the default printer and plotting statistics.

Project Planning

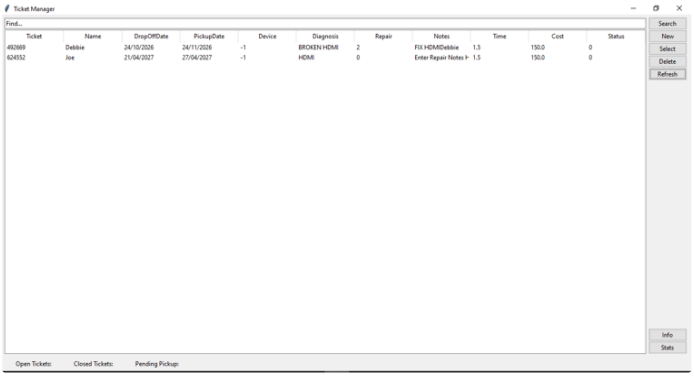
To plan the core functionality, a chart was made with transparency film and sticky notes. This allowed for easy adjustment of planning as the program was being built. Some initial research was conducted in order to select a database and how to build a GUI in python.



Application Overview

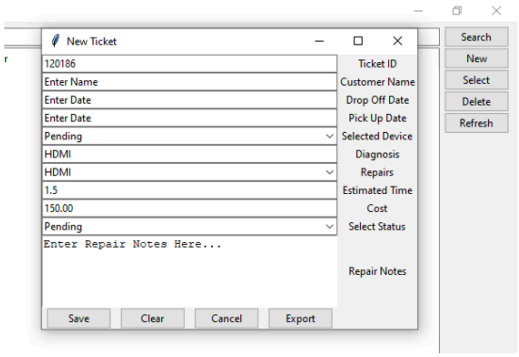
Application Layout:

The main window consists of a search bar and button, a column of buttons to create, edit and delete tickets. The main area of the window is filled with a tree view that displays the SQLite database.



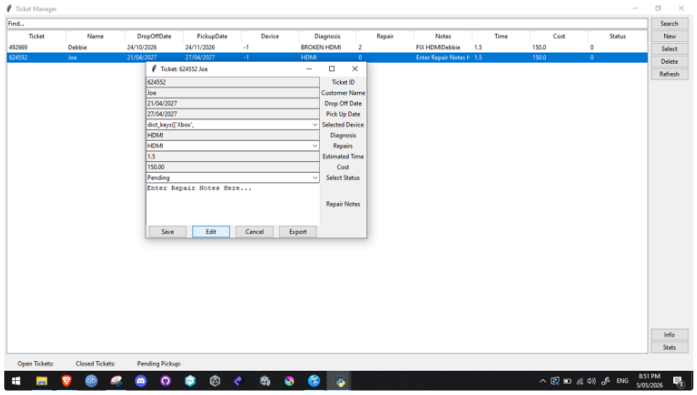
Creating a New Ticket:

1. Click on “New“
2. Enter the desired fields, dates must be formatted like so: dd/mm/yyyy , 01/12/2000
3. Click “Save“
4. Click “Refresh“



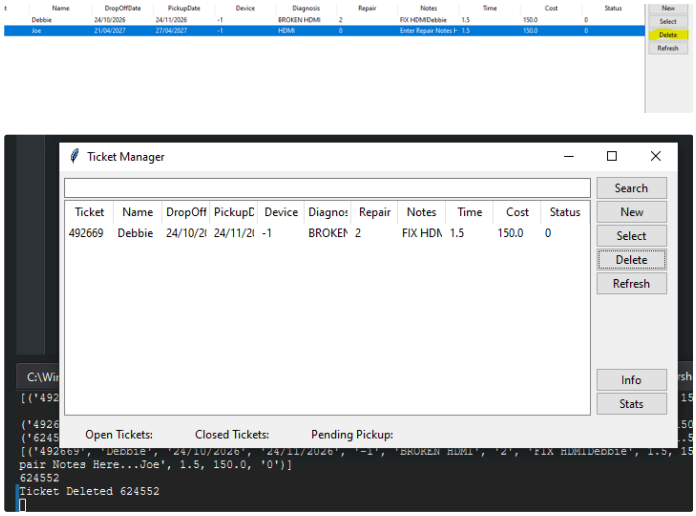
Editing an Existing Ticket:

1. Double click on the ticket you want to inspect
2. Click on “Edit“
3. Enter the desired fields, dates must be formatted like so: dd/mm/yyyy , 01/12/2000
4. Click “Save“ or Export if printing is desired
5. Click “Refresh“



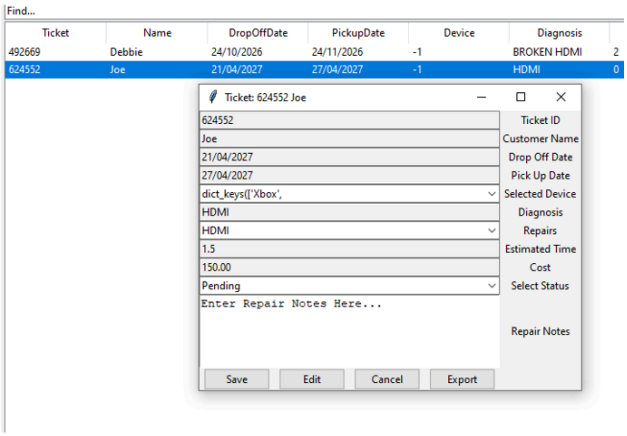
Deleting a Ticket:

1. Selected a ticket
2. Click on “Delete“



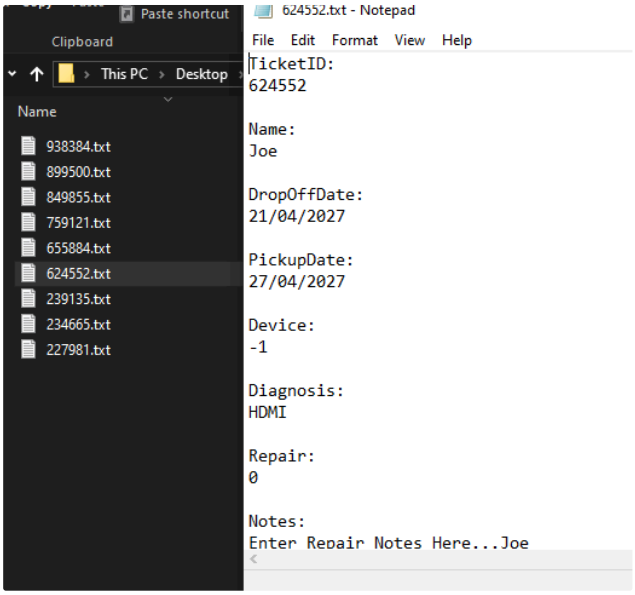
Select and View Ticket:

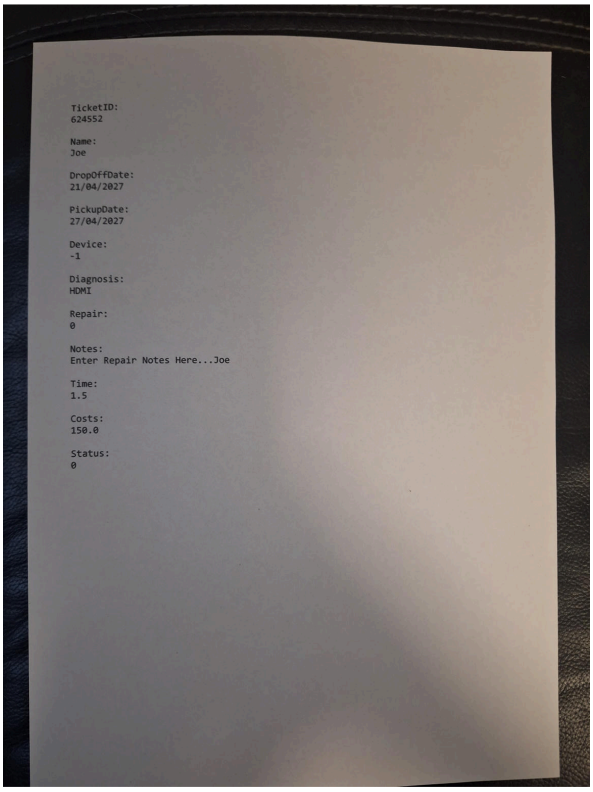
1. Double click on desired ticket



Printing a Ticket:

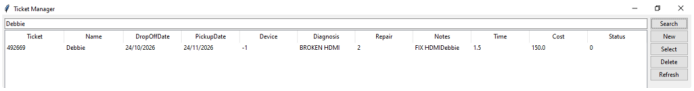
1. With a ticket open, click “Export“, this will save a text document with the ticket ID as the name and the send to your default printer (Only works on windows)





Search For a Ticket:

1. Enter any search term to find matching results, then click “Search“



Information Button: HALF IMPLEMENTED

Stats Button: HALF IMPLEMENTED

Technical Details

Tools

Kate was used to write the entire program

Kate - Get an Edge in Editing

Libraries

The libraries used in this program are as follows:

1. matplotlib : For plotting
2. numpy : For list management
3. sqlite3: database
4. tkinter: For entire GUI
5. pandas: For framing data like lists
6. defs: Local to Ticket Manager, holds all main definitions.

```
import matplotlib
import numpy as np
matplotlib.use('TkAgg')
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import (FigureCanvasTkAgg, NavigationToolbar2Tk)
import sqlite3
import tkinter as tk
from tkinter import ttk
import pandas as pd
import defs
```

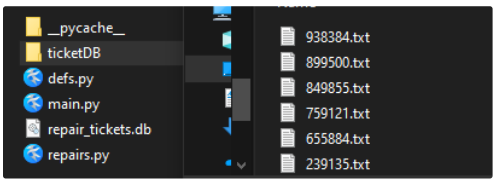
Program Structure

main.py starts the program

defs.py holds functions

The program saves tickets to an SQLite database that is easily accessed and managed through python.

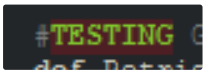
Exported tickets are saved to a subfolder with the relative Ticket ID as the name



Code Annotation

Code is annotated with “NOTE“, “TEST“, ”TESTING” or “TODO“, these are easily found and stand out among regular comments. Many editors can use these a references for easy viewing of tasks or notes.

This helps to go back to problem areas, label test code, describe a function or just organize code better for others who need to read it.

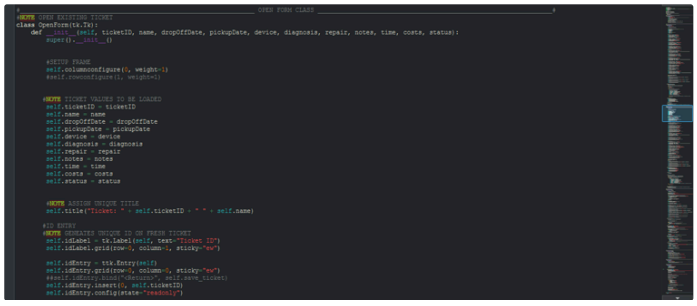


Main Classes

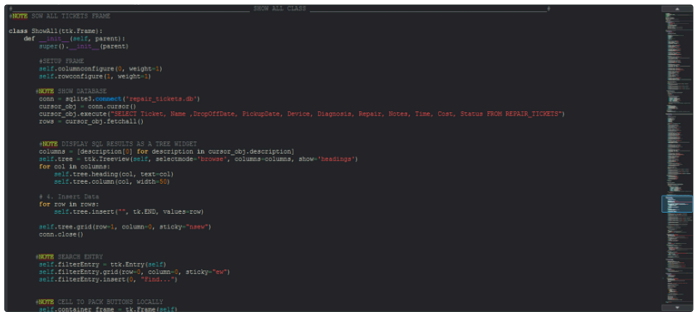
```
#NOTE INITIALISES APPLICATION ROOT AND SETS UP FRAMES
class Application(tk.Tk, object):
    def __init__(self):
        super().__init__()
        self.title("Ticket Manager")

        #NOTE MAKE FULL SCREEN BY DEFAULT, NOT THE SAME AS .attributes(-Fullscreen, bool)
        self.state("zoomed")

        #NOTE WEIGTH THE GRID TO SACLE WITH RATIO
        self.grid_rowconfigure(0, weight=1)
        self.grid_columnconfigure(2, weight=1)

        #NOTE START SHOW ALL FRAME CLASS
        showAll_Frame = ShowAll(self)
        showAll_Frame.grid(row=0, column=2, sticky="nsew", padx=5, pady=5)

        #NOTE START TOOLBAR STATS CLASS
        toolbar_Frame = ToolBarStat(self)
        toolbar_Frame.grid(row=3, column=2, sticky="sew", padx=5, pady=5)
```

Application class is the main Tkinter window and has nested frames inside

Create a new Ticket



Open ticket, takes in ticket values and assigns them at start



Retrieves all tickets from database and displays them on a Tkinter tree

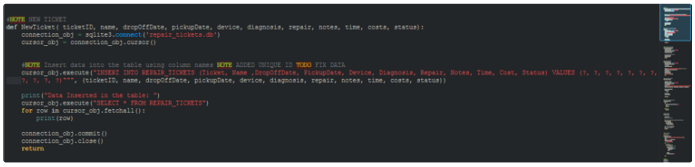
```
#_____ TOOLBAR STAT CLASS TODO _____#

class ToolBarStat(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent)

        #NOTE CELL TO PACK LOCALLY
        container_frame = ttk.Frame(self)
        container_frame.grid(row=1, column=1, sticky="ew")

        #NOTE OPEN TICKETS TODO
        self.openLabel = ttk.Label(container_frame, text="Open Tickets:")
        self.openLabel.pack(side=tk.LEFT, padx=20)


        #NOTE CLOSED TICKETS TODO
        self.closedLabel = ttk.Label(container_frame, text="Closed Tickets:")
        self.closedLabel.pack(side=tk.LEFT, padx=20)


        #NOTE PENDING PICKUP TODO
        self.pendingLabel = ttk.Label(container_frame, text="Pending Pickup:")
        self.pendingLabel.pack(side=tk.LEFT, padx=20)

#_____ INFO PANEL CLASS TODO _____#

class InfoPanel(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("InFormation")
        #NOTE INFORMATION PANEL TODO

        self.new_btn = ttk.Button(self, text="New", command= InputForm)
        self.new_btn.grid(row=0, column=0, sticky= "nw")


#_____ GRAPH CLASS TODO _____#
```

Not implemented, however was going to show status in main window and also display a handful of graphs

Main Definitions

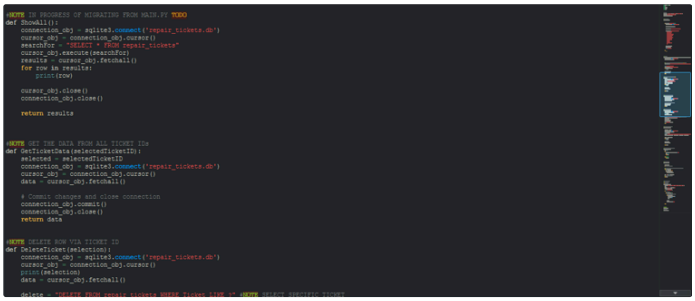
```
#NOTE CREATE DATABASE
def CreateSQLDatabase():

    if os.path.exists('repair_tickets.db') == False:
        connection_obj = sqlite3.connect('repair_tickets.db')

        cursor_obj = connection_obj.cursor()

        cursor_obj.execute("DROP TABLE IF EXISTS REPAIR_TICKETS")

        cursor_obj.execute("""CREATE TABLE REPAIR_TICKETS(
            TIcket VARCHAR(255),
            Name VARCHAR(255),
            DropOffDate VARCHAR(255),
            PickupDate VARCHAR(255),
            Device VARCHAR(255),
            Diagnosis VARCHAR(255),
            Repair VARCHAR(255),
            Notes VARCHAR(255),
            Time FLOAT,
            Cost FLOAT,
            Status VARCHAR(255))
            """)

        connection_obj.commit()
        connection_obj.close
        print("Database Created")

    else:
        return
```

Initialize database if it doesn't exist already



```
#NOTE NEW TICKET
def NewTicket( ticketID, name, dropOffDate, pickupDate, device, diagnosis, repair, notes, time, costs, status):
    connection_obj = sqlite3.connect('repair_tickets.db')
    cursor_obj = connection_obj.cursor()

    #NOTE Insert data into the table using column names NOTE ADDED UNIQUE ID TODO FIX DATA
    cursor_obj.execute("INSERT INTO REPAIR_TICKETS (Ticket, Name ,DropOffDate, PickupDate, Device, Diagnosis, Repair, Notes, Time, Cost, Status) VALUES (?, ?, ?, ?, ?, ?, ?,
    ?, ?, ?, ?)", (ticketID, name, dropOffDate, pickupDate, device, diagnosis, repair, notes, time, costs, status))

    print("Data Inserted in the table: ")
    cursor_obj.execute("SELECT * FROM REPAIR_TICKETS")
    for row in cursor_obj.fetchall():
        print(row)

    connection_obj.commit()
    connection_obj.close()
    return
```

Create new ticket, save received data to database



```
#NOTE IN PROGRESS OF MIGRATING FROM MAIN.PY TODO
def ShowAll():
    connection_obj = sqlite3.connect('repair_tickets.db')
    cursor_obj = connection_obj.cursor()
    searchFor = "SELECT * FROM repair_tickets"
    cursor_obj.execute(searchFor)
    results = cursor_obj.fetchall()
    for row in results:
        print(row)

    cursor_obj.close()
    connection_obj.close()

    return results


#NOTE GET THE DATA FROM ALL TICKET IDs
def GetTicketData(selectedTicketID):
    selected = selectedTicketID
    connection_obj = sqlite3.connect('repair_tickets.db')
    cursor_obj = connection_obj.cursor()
    data = cursor_obj.fetchall()

    # Commit changes and close connection
    connection_obj.commit()
    connection_obj.close()
    return data


#NOTE DELETE ROW VIA TICKET ID
def DeleteTicket(selection):
    connection_obj = sqlite3.connect('repair_tickets.db')
    cursor_obj = connection_obj.cursor()
    print(selection)
    data = cursor_obj.fetchall()

    delete = "DELETE FROM repair_tickets WHERE Ticket LIKE ?" #NOTE SELECT SPECIFIC TICKET
```

Show all is in the progress of migrating from main.py, get ticket data gets all data

```
#NOTE SEARCH FOR ROWS MATCHING TEXT
def SearchTicket(searchTerm):

    searchResults = searchTerm
    connection_obj = sqlite3.connect('repair_tickets.db')
    cursor_obj = connection_obj.cursor()
    print(searchResults)
    searchFor = "SELECT * FROM repair_tickets WHERE Ticket LIKE ? OR Name LIKE ? OR DropOffDate LIKE ? OR PickupDate LIKE ? OR Device LIKE ? OR Diagnosis LIKE ? OR Repair
    LIKE ? OR Notes LIKE ? OR Time LIKE ? OR Cost LIKE ? OR Status LIKE ?" #TODO ADD NEW VARIABLES
    cursor_obj.execute(searchFor, ('%' + searchResults + '%','%' + searchResults + '%','%' + searchResults + '%','%' + searchResults + '%','%' + searchResults + '%','%' +
    searchResults + '%','%' + searchResults + '%','%' + searchResults + '%','%' + searchResults + '%','%' + searchResults + '%'))

    results = cursor_obj.fetchall()
    for row in results:
        print(row)

    cursor_obj.close()
    connection_obj.close()

    return results


#NOTE VALIDATE DATE FORMATING USING DATETIME
def ValidateDate(date_Data):
    valid = bool
    try:
        dateObject = datetime.strptime(date_Data, '%d/%m/%Y')
        valid = True
        return valid

    except ValueError:
        return valid


#NOTE GENERATE UNIQUE ID
def GenerateUniqueID():

    random_numbers = []
```

Search ticket allows a wildcard search for items, validate date checks if the date format is valid

```
#NOTE GENERATE UNIQUE ID
def GenerateUniqueID():

    random_numbers = []
    #TODO CHECK AGAINST SQL DATABASE

    #NOTE Generate 2 unique 3 digit numbers and check if it is free to use
    while len(random_numbers) < 2:
        num = random.randint(100, 999)
        if num not in random_numbers:
            random_numbers.append(num)

    newID = ''.join(map(str, random_numbers))
    #NOTE RETURN NEW ID TO USER FORM
    return newID



#NOTE SEND DATA TO PRINTER
#TODO ADD PRINT WINDOW
#TODO ADD FORMATTING
def PrintTicket(dataToPrint):
    nameList = ["TicketID", "Name", "DropOffDate", "PickupDate", "Device", "Diagnosis", "Repair", "Notes", "Time", "Costs", "Status"]
    count = 0
    file_path = r"tickets\ID_"+dataToPrint[0]+".txt"
    if os.path.exists(file_path):
        print("file already exists")
    else:
        # create a file
        with open(file_path, 'w') as fp:
            # uncomment if you want empty file
            for item in dataToPrint:
                fp.write(nameList[count]+":\n")
                fp.write(item+"\n\n")
                count += 1

            fp.close()
            os.startfile(file_path, "print")
```

Generate ID generates a unique ID for each ticket, Print ticket prints the selected ticket and saves it as a text file.

Lessons Learned

The Good Stuff

Some of the lessons learned in this project are as followed:

1. Use Class parenting for easier time accessing functions from instances such as refresh.
2. How to use highlighted comments in Kate such as TODO, TEST, NOTE that helped keep track of all the important comments and reminders. I was easily able to see TODOs and work on them as I go, though there are still many left, its no where near the total I've used while making this application.
3. How to use SQL databases with sqlite3, which allowed for better access and management of more complex data sets with 10 categories.
4. Learned how to use tkinter to make a python GUI, learned how frames and windows work as well as how to pack multiple widgets to a single cell and use grids.
5. Refined my ability to work with larger lists and dictionaries, though there is still room to improve my mental map of them.
6. Learned to keep majority of variables local to classes and pass them to definitions in another file.

The Not So Good Stuff

Some of the things that didn't go so well were:

1. I didn't implement enough error checking.
2. Very limited cases of unit testing that often got overwritten.
3. Didn't finish implementing the graph viewer. I wanted device types, ticket counts and repair types over a selected period.
4. Didn't finish the print dialogue, however basic functions are still working.
5. Didn't finish information window.
6. Combo boxes don't reflect corresponding selection in tree view and need remapping

Future Direction

The plan for this application is to fix the problems mentioned above, and then work on getting theming to work with OS settings with tkinter ttk which tries to match the window to a theme or to the OS system theme.

Additionally I would like to adjust the ticket and edit forms layout and parent them to the main class so functions can be called across instances better.

Finally Id like to expand on the devices and repairs listed in the program as well as calculate GST on export of the ticket.